Programming Project: Robot Maze Escape Using a Graph and BFS in Java
Course: Data Structures and Algorithms
Programming Language: Java
Project Type: Individual/Group Programming Assignment

1. Problem Description

Scenario: Robot Escape from a Dangerous Building

Imagine that a rescue robot is trapped inside a large building after an emergency. The building contains many rooms, corridors, walls, and dead ends. The robot has access to a digital map of the building that shows the walls and open paths. However, it does not know the shortest route to the exit.

Your task is to develop an intelligent maze-solving program that allows the computer to automatically find the shortest path using Breadth-First Search (BFS) and guide the robot safely to the exit without hitting any walls.

You are required to develop a **Robot Maze Escape Challenge using Java**. The main purpose of this project is to understand how a **Graph using an Adjacency Matrix** and the **Breadth-First Search (BFS)** algorithm can solve a pathfinding problem.

In this project:

- The building will be represented as a maze containing walls and open paths.
- Each open cell will represent a vertex in a graph.
- Connections between adjacent open cells will represent edges.
- The graph must be stored using an adjacency matrix.
- The computer must use BFS to calculate the shortest path from the robot's current position to the exit.
- After finding the path, the robot must automatically move through the maze, following the calculated route.
- The robot must avoid walls and reach the exit successfully.
- The program must generate a different random maze whenever the user restarts the game.

Important: The user will not manually control the robot's movement. The computer must calculate the escape route using BFS and automatically move the robot along that path.

The game must use a graphical window created with **Java Swing**. You must also use the console to compile and run your Java program.

2. Java Packages and Classes

You must use standard Java libraries only. No external libraries are allowed.

Import the following packages and classes:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.util.Random;
```

These libraries will be used for:

- **javax.swing:** Creating the graphical window, displaying the maze, and using a timer to animate the robot.
- **java.awt:** Drawing the maze, walls, robot, exit, and shortest path using different colors.
- **java.awt.event:** Reading keyboard input and controlling game actions.
- **java.util.Random:** Generating different random maze layouts.

You must create a Java file named **MazeSolver.java**.
Your program must contain a class named MazeSolver that extends JPanel.

```
public class MazeSolver extends JPanel {

   // Declare variables and methods here

}
```

You must also include a main() method to create the game window and start the program.

3. Graph and Adjacency Matrix Requirements
You must implement your own Graph using an Adjacency Matrix.
You are not allowed to use Java's built-in graph libraries or external graph libraries to represent the maze.
Each open cell of the maze must represent a vertex. Two vertices must be connected by an edge if their corresponding cells are directly adjacent (up, down, left, or right) and neither cell is a wall.

Use the following variables:

```
static final int SIZE = 21;
static final int N = SIZE * SIZE;

static int[][] maze = new int[SIZE][SIZE];
static int[][] graph = new int[N][N];
```

The maze may follow these rules:

- maze[row][col] = 1: The cell represents a wall.
- maze[row][col] = 0: The cell represents an open path.

The adjacency matrix may follow these rules:

- graph[i][j] = 1: A direct connection exists between vertex i and vertex j.
- graph[i][j] = 0: No direct connection exists between the two vertices.
- The graph must be **undirected**, meaning connections work in both directions.

For example:

```
graph[22][43] = 1;
graph[43][22] = 1;
```

This means vertex 22 is connected to vertex 43, and the robot can move between these two vertices.

You may convert each maze cell into a vertex number using the following formula:
Vertex Number = Row × SIZE + Column

For example, when SIZE = 21:

- Cell (1,1) represents vertex 22.
- Cell (2,1) represents vertex 43.
- Cell (1,2) represents vertex 23.

Use the following method:

```
static int vertex(int row, int col) {
   return row * SIZE + col;
}
```

Required Methods
Your program must implement methods that perform the following tasks:

Method	**Required Task**
vertex()	Convert row and column positions into a vertex number
generateMaze()	Generate a random maze containing walls, open paths, turns, and dead ends
createGraph()	Create the adjacency matrix from the generated maze
findShortestPath()	Use BFS to find the shortest path from the robot to the exit
startRobot()	Start the robot's automatic movement along the calculated path
resetGame()	Generate a new maze and return the robot to the starting position
paintComponent()	Draw the maze, walls, robot, exit, and shortest path
main()	Create the Swing window and start the program

You may create additional methods if needed.

BFS Requirements
You must implement the **Breadth-First Search (BFS)** algorithm yourself.
Use the following arrays:

```
boolean[] visited = new boolean[N];
int[] parent = new int[N];
int[] queue = new int[N];
```

These arrays have different purposes:

- visited[]: Records whether a vertex has already been visited.
- parent[]: Records the previous vertex used to reach each vertex.
- queue[]: Stores vertices waiting to be processed by BFS.

Use front and rear variables to manage the queue.

BFS must start from the robot's current position and continue searching until the exit is found or all reachable vertices have been explored.
After finding the exit, use the parent[] array to reconstruct the shortest path.
Important: You must not manually write or hardcode the robot's escape route. The computer must calculate the path using BFS for each generated maze.

4. Maze Generation and Appearance
Create a square maze containing **21 rows and 21 columns**, including the boundary walls.
The maze must be generated randomly instead of using a fixed layout.
Maze Generation Requirements
The program must:

- Generate a maze containing walls, open paths, multiple turns, and dead ends.
- Ensure that the robot has at least one valid route from the starting position to the exit.
- Keep the outer boundary closed with walls.
- Create a different maze whenever the user presses R.
- Rebuild the adjacency matrix after generating each new maze.
- Allow BFS to calculate a new shortest path for every maze.

You may use a randomized depth-first maze-generation technique with an array-based stack to create the maze.
The maze-generation algorithm and the BFS pathfinding algorithm must be treated as separate operations.

Game Appearance
Use the following colors:

Game Element	**Color**
Background and open paths	Black
Walls	Gray
Robot	Blue
Exit	Green
Shortest path	Yellow

The robot must appear as a small robot-like object rather than a simple square. You may draw the robot using basic Java graphics, including a head, eyes, body, and antenna.
The robot must initially appear near the upper-left corner of the maze.
The exit must appear near the lower-right corner.

Display the following information below the maze:

- Project title: **MAZE SOLVER - BFS**
- Keyboard instructions.
- Current status messages.
- Number of steps in the shortest path.

5. Keyboard Controls
The user must be able to control the program using the following keys:

Key	Action
B	Execute BFS and find the shortest path
P	Start the robot's automatic movement
R	Generate a new random maze and restart
Q	Quit the program

Important: The user must not need to press Enter after pressing a key.
The robot must not be controlled manually using W/A/S/D or arrow keys.
The computer must determine the robot's movement based on the shortest path calculated by BFS.

6. Game Rules and Conditions
Condition 1: Starting the Program
When the program starts, it must automatically generate a random maze.
The robot must appear at the starting position, and the exit must appear at the destination.
The robot must remain stationary until the user requests BFS pathfinding and starts the movement.
Display the message:
Press B to find the shortest path.

Condition 2: Generating a Random Maze
The program must create a random maze with walls, open paths, turns, and dead ends.
The maze must contain at least one valid path from the starting position to the exit.
The program must not use the same fixed maze layout every time.
After generating the maze, the program must create the corresponding adjacency matrix.

Condition 3: Finding the Shortest Path Using BFS
When the user presses **B**, the computer must execute BFS.
BFS must search the graph using the adjacency matrix and calculate the shortest path from the robot's current position to the exit.
The calculated path must be highlighted in yellow.
Display a message similar to:
BFS found shortest path: 48 steps. Press P.
The number of steps must be calculated automatically and may differ for each maze.
Important: The highlighted route must be calculated using BFS, not manually defined.

Condition 4: Starting the Robot
When the user presses **P**, the robot must automatically begin moving along the shortest path calculated by BFS.
The robot must move one cell at a time.

Use a Java Swing Timer to create a short delay between movements so that the user can watch the robot travel through the maze.
The robot must follow the calculated path without any manual directional input.
Display the message:
Robot moving...

Condition 5: Avoiding Walls
The robot must move only through open cells connected by valid graph edges.
The robot must not pass through walls or move outside the maze.
The movement must follow the shortest path calculated by BFS.

Condition 6: Pressing P Before BFS
If the user presses **P** before pressing B, the robot must not start moving.
Display the message:
Press B first to calculate the path.
The robot must remain at its current position.

Condition 7: Reaching the Exit
When the robot reaches the exit, the automatic movement must stop.
Display a success message such as:
Maze Solved Successfully! Robot escaped!
The program must also display the number of steps used to reach the exit.
The program must remain open after the robot reaches the exit.

Condition 8: Restarting the Program
When the user presses **R**, the program must:

- Stop any ongoing robot movement.
- Generate a new random maze.
- Rebuild the adjacency matrix.
- Return the robot to the starting position.
- Clear the previous shortest path.
- Reset the path length and movement progress.
- Display the message:

Press B to find the shortest path.
The user must be able to repeat the maze-solving process without closing the program.

Condition 9: Recalculating the Shortest Path
For every newly generated maze, the program must execute BFS again when the user presses B.
The new path must be calculated based on the new maze layout.
The program must not reuse the shortest path from a previous maze.

Condition 10: No Available Path
Although the maze-generation algorithm should ensure that a valid escape route exists, your BFS implementation must still handle the situation where no path is found.

If the exit is unreachable, display:
No path to the exit!
The robot must not begin moving without a valid path.

Condition 11: Continuing the Program
The program must continue running after the robot successfully escapes.
The user must be able to press R to generate another maze and repeat the process as many times as desired.
There must be no fixed limit on the number of mazes the program can solve.

Condition 12: Quitting the Program
The program must close when the user presses **Q**.
The user must also be able to close the program using the window's close button.

7. Program Execution
Your program must be saved as **MazeSolver.java**.
Open a terminal or command prompt in the folder containing the Java file.

Compile the program:
javac MazeSolver.java
Run the program:
java MazeSolver

A Swing window must open and display the randomly generated maze.
Press **B** to calculate the shortest path using BFS.
Press **P** to watch the robot automatically move along the path and escape.
Press **R** to generate a new random maze.
Press **Q** to quit the program.
Important: The console must be used to compile and launch the program. The actual maze and robot movement must appear in the Swing window.

8. Testing Requirements
Before submitting your project, test all of the following:

- The program opens a Swing window successfully.
- A random 21 × 21 maze is generated.
- The maze contains walls, open paths, turns, and dead ends.
- The robot and exit are displayed in the correct positions.
- The graph is represented using an adjacency matrix.
- The adjacency matrix correctly represents connections between open cells.
- Pressing B executes BFS and calculates the shortest path.
- The shortest path is highlighted in yellow.
- The shortest-path length is displayed correctly.
- Pressing P before B does not start the robot.
- Pressing P after B starts automatic robot movement.

- The robot moves one cell at a time along the calculated path.
- The robot does not pass through walls.
- The robot successfully reaches the exit.
- The movement stops when the robot reaches the exit.
- Pressing R generates a different maze and resets the robot.
- BFS calculates a new path for the newly generated maze.
- The program can solve multiple mazes without restarting Java.
- Pressing Q closes the program.

9. Submission Requirements

Submit your project with the following files:

I. **Java Source Code**
   Submit MazeSolver.java, containing your complete program.

II. **Video Recording**
   Submit a complete screen recording demonstrating your program and explaining the source code.

III. **Short Report (PDF)**

- Your report must explain:
- What a graph is and why it is used in this project.
- How maze cells are represented as vertices.
- How adjacent open cells are connected by edges.
- How the adjacency matrix is created and updated.
- How the random maze is generated.
- How BFS finds the shortest path.
- How the queue, visited array, and parent array are used.
- How the shortest path is reconstructed.
- How the robot automatically follows the calculated path.
- How Java Swing is used to display and animate the robot.
- Your program must compile and run successfully.

You must implement your own adjacency matrix and BFS algorithm. Using a built-in graph library, hardcoding the shortest path, or manually controlling the robot instead of implementing automatic pathfinding will not satisfy the project requirements.

Failure to follow the project instructions may result in grade deductions.

10. Video Recording Requirement

You must submit a **screen recording demonstrating your complete program from beginning to end**.

Start the recording by showing how you compile and run the Java program using the command prompt or terminal.

Then demonstrate the following activities:

1. Start the program and show the randomly generated maze, robot, walls, and exit.
2. Explain the starting position and exit location.
3. Press B to execute BFS and display the shortest path.
4. Show the shortest path highlighted in yellow and explain the displayed number of steps.

5. Press P to start the robot's automatic movement.
6. Demonstrate how the robot follows the calculated path without manual control.
7. Show the robot successfully reaching the exit.
8. Press R to generate a new random maze.
9. Demonstrate that the new maze has a different layout.
10. Press B again to calculate a new shortest path.
11. Press P again and demonstrate the robot escaping from the new maze.
12. Press Q to quit the program.

You must also **explain your source code clearly**, including the purpose of each imported package, class, method, variable, important statement, and the graph algorithms used in the project.
In particular, you must explain:

- How the maze[][] array represents walls and open paths.
- How the graph[][] adjacency matrix represents the graph.
- How row and column positions are converted into vertex numbers.
- How the random maze-generation algorithm works.
- How the createGraph() method creates connections between vertices.
- How the BFS algorithm explores vertices using the queue.
- How visited[] prevents repeated visits.
- How parent[] is used to reconstruct the shortest path.
- How the robot follows the path calculated by BFS.
- How the Swing Timer controls the robot's movement.
- How paintComponent() draws the maze, robot, exit, and shortest path.
- How pressing R generates a new maze and resets the program.

Explain how your program works step by step.
Important: Simply running the program without explaining the code is not sufficient. Incomplete, unclear, incorrect, or poor explanations will result in point deductions.

11. Additional Instructions

- Each student must submit their work separately.
- Your program must use an adjacency matrix to represent the graph.
- You must implement BFS to calculate the shortest path.
- The robot must move automatically based on the calculated path.
- Your program must generate a new random maze when restarted.
- Your code must be properly organized and readable.
- You must be able to explain how your code works.
- Your source code, video recording, and PDF report must be submitted before the deadline.
- Failure to follow the instructions will result in grade deductions.

Note: Please review the sample output of the Robot Maze Escape Challenge and the sample video demonstration from another project for reference.